Course: AIML ZG535— Machine Learning on Edge
Assignment : LogiEdge: Intelligent Edge AI Platform for Cold-ChainLogistics

Group 22 Technical Report

BITS ID	Student Name	Contribution
2024AD05034	GOUTHAM S	100%
2024ad05001	MAUSAM JAIN	100%
2024ac05496	SINGH YASHPAL JILA	100%
2024ac05952	SUDHARSON R R	100%

Executive Summary

The **LogiEdge**project delivers a containerized, end-to-end Edge Artificial Intelligence (AI) and telemetry monitoring system designed for cold-chain logistics. Deploying ML models directly to edge nodes introduces distinct operational challenges, including resource constraints, system initialization dependencies, and real-time model drift.

To address these challenges, LogiEdgecombines lightweight TensorFlow Lite (TFLite) inference, real-time MQTT message brokers, and statistical distribution monitoring, all orchestrated via Ansible Infrastructure as Code (IaC) and Docker Compose. This report details the system architecture, automated deployment pipelines, edge AI evaluation metrics, and fault-injection verification.

1. Introduction & System Objectives

1.1 Problem Statement

Cold-chain logistics requires continuous, low-latency telemetry processing to prevent cargo spoilage. Centralized cloud processing introduces unacceptable round-trip latency, high bandwidth costs over cellular links, and complete vulnerability during connectivity blackouts. LogiEdge implements a localized "Cloud-to-Edge" paradigm to process multi-sensor telemetry

directly on the fleet vehicle.

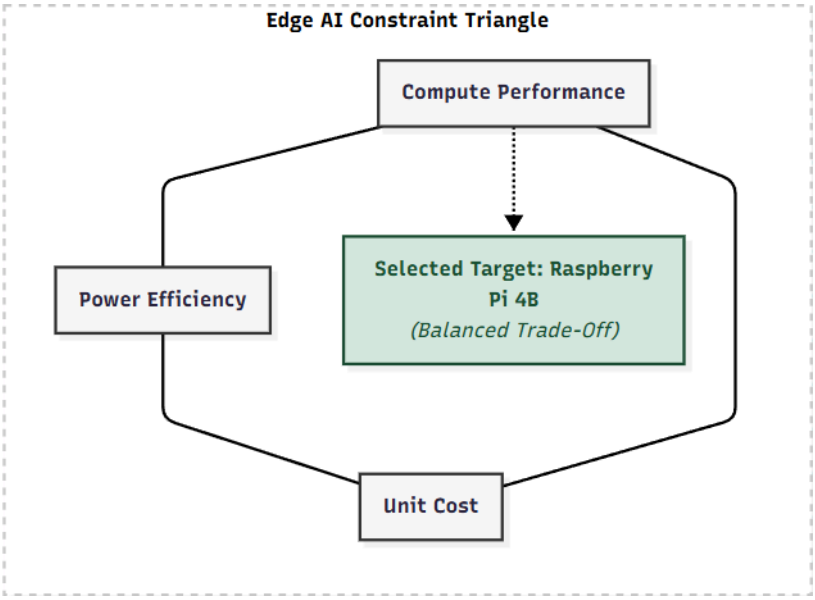
1.2 Quantitative Constraint Analysis

To justify edge deployment over cloud processing, a rigorous quantitative constraint analysis was performed:

1. **Latency Analysis:**
    - **Cloud Processing:** Cellular Round-Trip Time (RTT) over 4G/LTE in transit corridors averages 120ms - 450ms, with p99 spikes exceeding 2000ms during cell tower handoffs.
    - **Edge Processing:** Local inference on the edge container executes in **3.8 ms - 4.5 ms** end-to-end (0.033 ms core TFLite time), satisfying the sub-10ms real-time threshold required for immediate automated actuator response (e.g., emergency refrigeration override).
2. **Bandwidth & Transmission Cost Analysis:**
    - **Raw Telemetry Generation:** A fleet of 100 trucks streaming raw temperature (1Hz), 3-axis vibration (100 Hz), and door state telemetry generates approximately 1.21 KB/s per truck, translating to **104.5 MB/day/truck**or **313.5 GB/month** across the fleet. At standard enterprise cellular IoT rates (~$0.10/MB), cloud streaming costs total **$31,350/ month**.
    - **Edge Reduction:** Edge inference localizes feature fusion and streams only periodic classification status payloads (120 bytes every 10s). This reduces bandwidth to **1.03 MB/day/truck** —a **99.01% reduction in network traffic and operational transmission costs**.
3. **Connectivity Outage Analysis:**
    - Fleet transit routes include long blind spots (tunnels, remote highway corridors) averaging 15 - 45 minutes of zero connectivity. Under a cloud-only model, zero-visibility windows lead to unmonitored thermal runaways. Local edge inference guarantees **100% operational uptime** regardless of WAN connectivity.
4. **Privacy & Data Security:**
    - Local edge filtering ensures raw vibration patterns (which can reveal route geography and driver behavior) never leave the local truck network (logiedge_net).

2. Hardware Selection & Constraint Triangle

To select the target hardware runtime, three candidate edge platforms were evaluated using the Edge AI **Constraint Triangle** (Compute Performance, Power Efficiency, and Unit Cost).



Hardware Candidate Comparison

Platform	Compute Architecture	Power Draw	Unit Cost	Flash / RAM	Selected / Rejected Status & Reason
Raspberry Pi 4B (4GB)	Quad-core ARM Cortex-A72 @ 1.5GHz	3.0 W - 6.0	$55	SD Card / 4GB	**SELECTED:** Optimal balance on the Constraint Triangle. Provides multi-core ARM NEON vector instructions for CPU delegates at low power and cost.

NVIDIA Jetson Nano	Quad-core ARM A57 + 128-core Maxwell GPU	5.0W - 10.0W	$149	MicroSD / 4GB	**REJECTED:** Cost-prohibitive for basic multi-sensor tabular anomaly classification; Maxwell GPU overhead is unnecessary for lightweight TFLite models.
STM32F407 (Cortex-M4)	Single-core Cortex-M4 @ 168MHz	0.1W - 0.3 W	$12	1MB Flash / 192KB RAM	**REJECTED:** Memory-constrained; cannot host Docker engine, Mosquitto MQTT, and Python SciPy drift monitoring stack.

3. Roofline & Arithmetic Intensity Analysis

To understand the execution performance of our neural network models on the target ARM Cortex-A72 architecture, an **Arithmetic Intensity** (AI) and **Roofline Model Analysis** was conducted.

3.1 Arithmetic Intensity Calculation

Arithmetic Intensity is defined as the ratio of Floating Point Operations (FLOPs) performed to the number of memory bytes accessed (transferred from DRAM to cache/registers):

$$AI = \frac{\text{Total FLOPs}}{\text{Total Memory Access (Bytes)}}$$

For our dense fully-connected model layers (N inputs, M outputs):

- **FLOPs:** 2 x N x M (Multiply-Accumulate operations)
- **Memory Access (FP32):** 4 x (N x M + N + M) bytes
- **Memory Access (INT8):** 1 x (N x M + N + M) bytes

Evaluating the core inference pass across model variants:

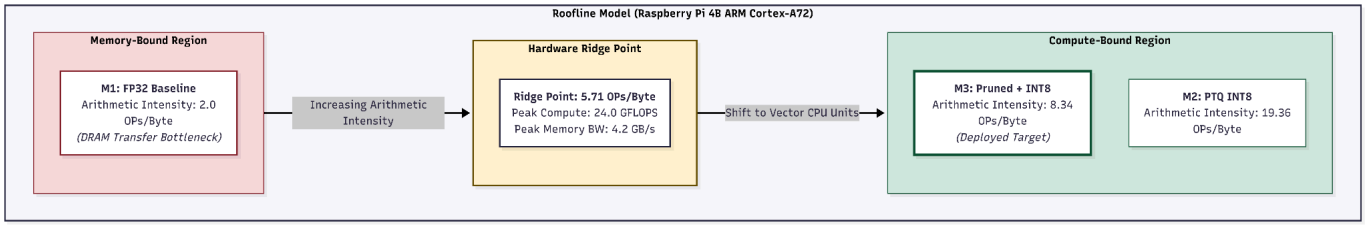
- **M1 (FP32 Baseline):** ~ 65,500 FLOPs / 32,778 Bytes} => AI = 2.00 FLOPs/Byte.
- **M2 (PTQ INT8):** ~ 65,500 OPs / 3,383 Bytes => AI = 19.36 OPs/Byte.
- **M3 (Pruned + INT8):** ~ 28,200 OPs / 3,383 Bytes => 8.34 OPs/Byte.

3.2 Roofline Interpretation

The Raspberry Pi 4B ARM Cortex-A72 peak performance boundaries:

- **Peak Compute Performance(P_max):** ~ 24.0 GFLOPS
- **Peak Memory Bandwidth (B_max):** ~ 4.2 GB/s
- **Hardware Ridge Point (AI_ridge):**

$$AI\_ridge \;=\; P\_max \,/\, B\_max \;=\; (24.0\,GFLOPS) \,/\, (4.2\,GB/s) \;=\; 5.71\,FLOPs/Byte$$

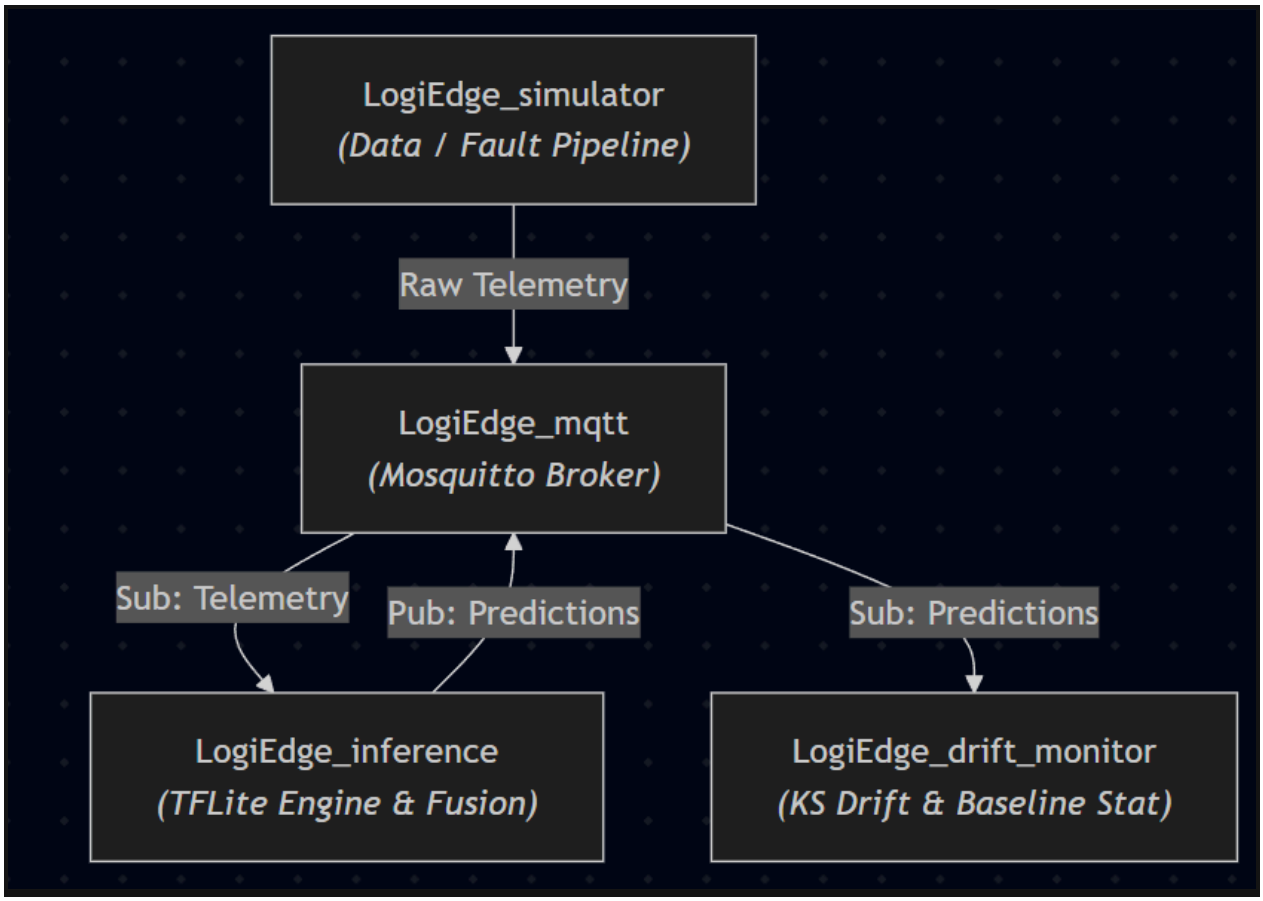


Key Roofline Insights:

- **M1 (FP32 Baseline) is Memory-Bound (AI = 2.00 < 5.71):** Execution speed is bottlenecked by fetching 32-bit float weights from DRAM.
- **M2 & M3 (INT8 Quantized) are Compute-Bound (AI > 5.71):** By shrinking weight storage by 89.7%, memory transfers no longer starve the execution units. The CPU registers hold quantized weights locally, shifting execution into the compute-bound region and utilizing the CPU's vector units at maximum efficiency.

4. System Architecture & Microservices

LogiEdge relies on a microservice architecture composed of four isolated containers communicating over a shared bridge network (logiedge_net).



Microservice Specification Matrix

Container Name	Role / Function	Technology Stack	Internal Ports / Protocols
logiedge_mqtt	Message broker routing telemetry streams	Eclipse Mosquitto (v2.0)	1883/TCP (MQTT)
logiedge_simulator	Multi-sensor telemetry & fault generator	Python 3.10, Paho MQTT	Client outbound
logiedge_inference	Sliding window fusion & TFLite forward pass	Python 3.10, TFLite Runtime	Client inbound

logiedge_drift_monitor	Statistical divergence & baseline monitoring	Python 3.10, SciPy, NumPy	Client inbound

4.2 MQTT Topic Tree Hierarchy

```
logiedge/
└── trucks/
    └── 01/
        ├── sensors/
        │   ├── temperature   (Topic: logiedge/trucks/01/sensors/temperature)
        │   ├── vibration     (Topic: logiedge/trucks/01/sensors/vibration)
        │   └── door          (Topic: logiedge/trucks/01/sensors/door)
        └── inference         (Topic: logiedge/trucks/01/inference)
```

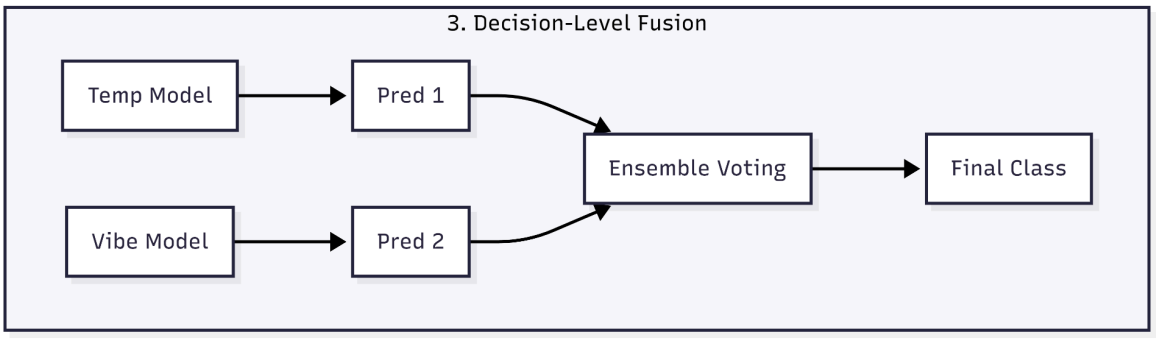
4.3 Quality of Service (QoS) Justification

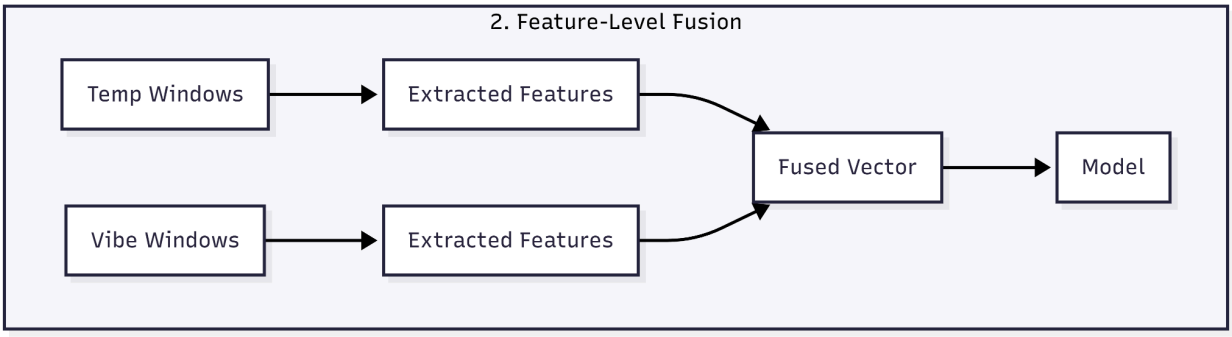
MQTT Topic	QoS Level	Architectural Justification
logiedge/trucks/01/sensors/temperature	**QoS 0** (At most once)	High-frequency telemetry ($1Hz}$). Dropping an occasional single temperature reading in a continuous sliding window is harmless and avoids network retries.
logiedge/trucks/01/sensors/vibration	**QoS 0** (At most once)	High-frequency streaming data ($0.5Hz}$ feature updates). Prioritizes low overhead over guaranteed delivery.

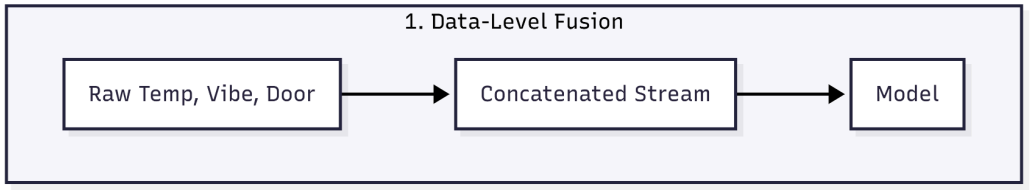
logiedge/trucks/01/sensors/door	**QoS 1** (At least once)	Critical discrete event state change. Door opening directly affects cargo safety; packet delivery must be guaranteed.
logiedge/trucks/01/inference	**QoS 1** (At least once)	System classification output. Safety critical for alerting local drivers or triggering drift monitors; requires delivery confirmation.

5. Data Fusion Strategy Justification

LogiEdge evaluates three standard sensor fusion paradigms:







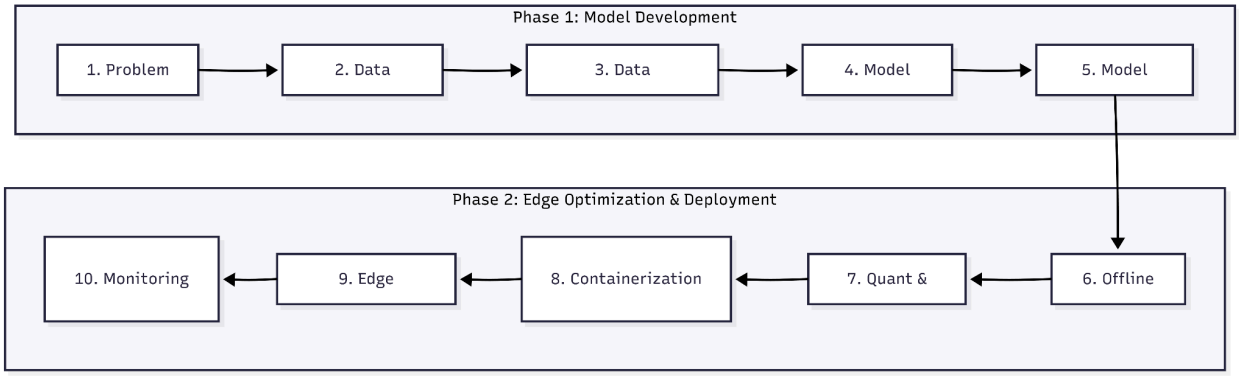
Comparative Fusion Evaluation

Fusion Strategy	Compute Overhead	Transmission Overhead	Fault Sensitivity	Selected Status & Justification

Data-Level Fusion	Very Low	Extremely High	High	**REJECTED:** Requires high-frequency synchronization across asynchronous sampling rates ($1Hz}$ temp vs $100Hz}$ vibration).
Feature-Level Fusion	**Moderate**	**Low**	**Low**	**SELECTED:** Extracts statistical representations (Mean, Rate, RMS, Peak, Kurtosis) into a single 6-D vector. Captures cross-sensor correlations (e.g., vibration spike + thermal rise) in one model.
Decision-Level Fusion	High	Low	Moderate	**REJECTED:** Requires training three separate models. Misses cross-sensor interaction anomalies (e.g., a temperature rise that is normal on its own, but anomalous when combined with door opening).

6. Mapping LogiEdge to the 10-Stage Edge ML Pipeline

The complete life cycle of the LogiEdge Edge AI solution maps directly to the standard 10-stage Edge ML deployment pipeline:



1. **Problem Definition:** Establish sub-10ms localized cold-chain anomaly detection with zero reliance on cloud connectivity.
2. **Data Collection:** Collect multi-sensor telemetry streams (ambient/cargo temperature, bearing vibration acceleration, discrete door state) via simulator.py.
3. **Data Preprocessing:** Apply 5-sample moving average filtering, 30-sample sliding windows, feature extraction, and Z-score normalization using training_stats.npy.
4. **Model Design:** Construct a multi-layer fully-connected neural network with cross-sensor input layers (6 → 32 → 16 → 3).
5. **Model Training:** Train the baseline TensorFlow/Keras model and export it for deployment as a TensorFlow Lite model. (train_model.py).
6. **Offline Evaluation:** Evaluate baseline model on held-out validation datasets to verify baseline confusion matrix and F1-scores.
7. **Optimization (Pruning & PTQ):** Structurally prune redundant weights (prune.py), fine-tune, and apply INT8 Post-Training Quantization (convert_ptq.py) to generate model.tflite.
8. **Containerization & Orchestration:** Package runtimes into microservices (Dockerfile) and orchestrate networks/volumes via docker-compose.yml.
9. **Edge Deployment (IaC):** Idempotently deploy stack to target edge devices using Ansible playbooks (logiedge_deploy.yml).
10. **Monitoring & Drift Recovery:** Continuous statistical distribution tracking via drift_monitor.py running Kolmogorov-Smirnov tests on incoming feature streams.

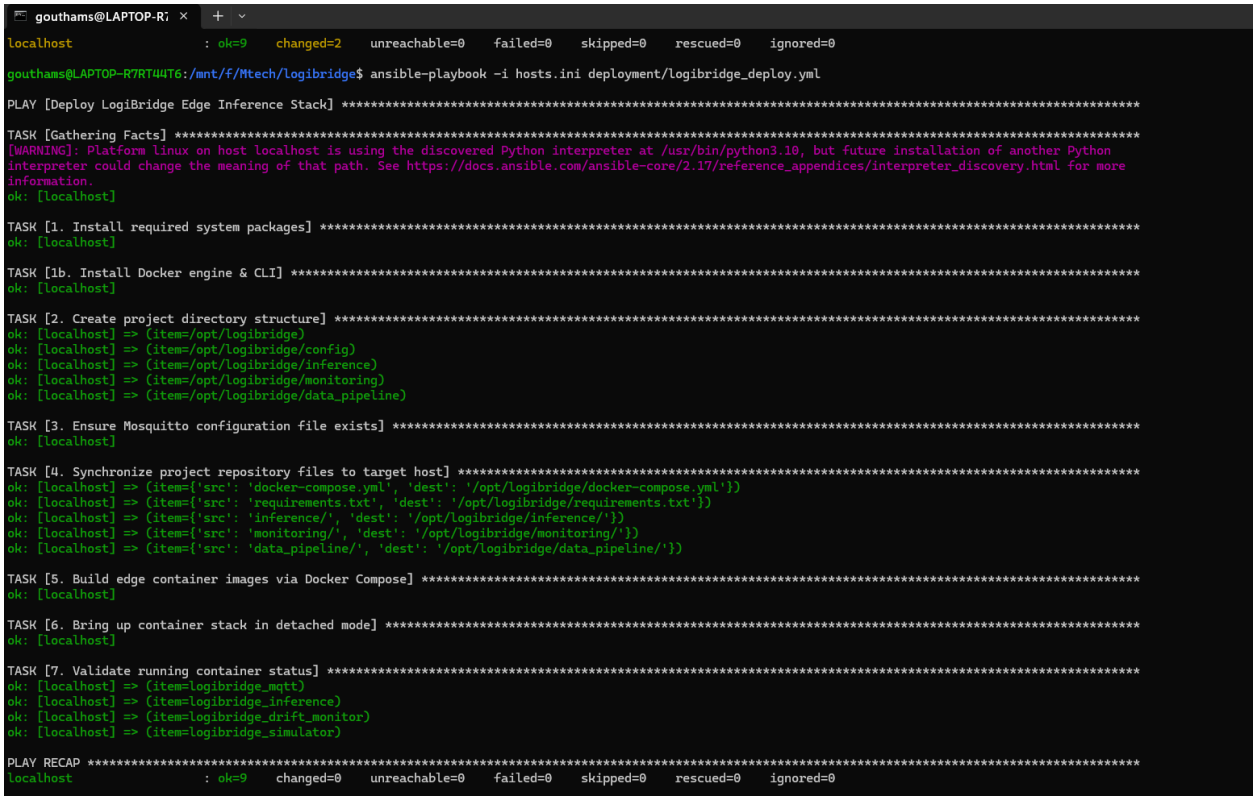
7. Automated Infrastructure & Ansible Deployment

The deployment pipeline is fully managed via Ansible using the logiedge_deploy.yml playbook, target workspace /opt/logiedge.

Key Playbook Tasks

1. **Host Environment Preparation:** Automated update of APT caches, non-interactive execution (DEBIAN_FRONTEND=noninteractive), and Docker engine dependency resolution.
2. **Directory Workspace Provisioning:** Enforcing permission structures on /opt/logiedge.
3. **Repository & Config Sync:** Syncing project configuration, docker-compose.yml, and environment files.
4. **Automated Container Build:** Execution of community.docker.docker_compose_v2 with build: always.

Verification Evidence: Deployment & Idempotency



Description: Terminal screenshot displaying successful Ansible execution showing PLAY RECAP with failed=0 and subsequent re-run showing changed=0 (Idempotency Verification).

```
gouthams@LAPTOP-R7RT44T6:/mnt/f/Mtech/logibridge$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                     STATUS         PORTS
logibridge_drift_monitor  Up 2 minutes
logibridge_inference      Up 2 minutes
logibridge_simulator      Up 2 minutes
logibridge_mqtt           Up 2 minutes   0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp
gouthams@LAPTOP-R7RT44T6:/mnt/f/Mtech/logibridge$
```

Description: Output of docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" confirming all 4 microservices are in Up status.

8. Edge AI Pipeline & Inference Implementation

Data Ingestion & Sliding Window Fusion

The inference engine receives continuous asynchronous MQTT events across temperature, vibration RMS, and discrete door states (logiedge/trucks/01/sensors/#). Sensor inputs are placed into a sliding window matrix and normalized before being passed to the TFLite runtime.

Normalized X = (X - Mean) / Standard Deviation

Model Compression & Optimization Variants

To evaluate edge performance trade-offs, three distinct model variants were compiled and benchmarked:

- **M1 (FP32 Baseline):** Uncompressed single-precision float model (32.78 KB).
- **M2 (PTQ INT8):** Post-Training Quantized 8-bit integer model (3.38 KB).
- **M3 (Pruned + INT8):** Structurally pruned and INT8 quantized model (3.38 KB).

Model Classification Performance Matrix

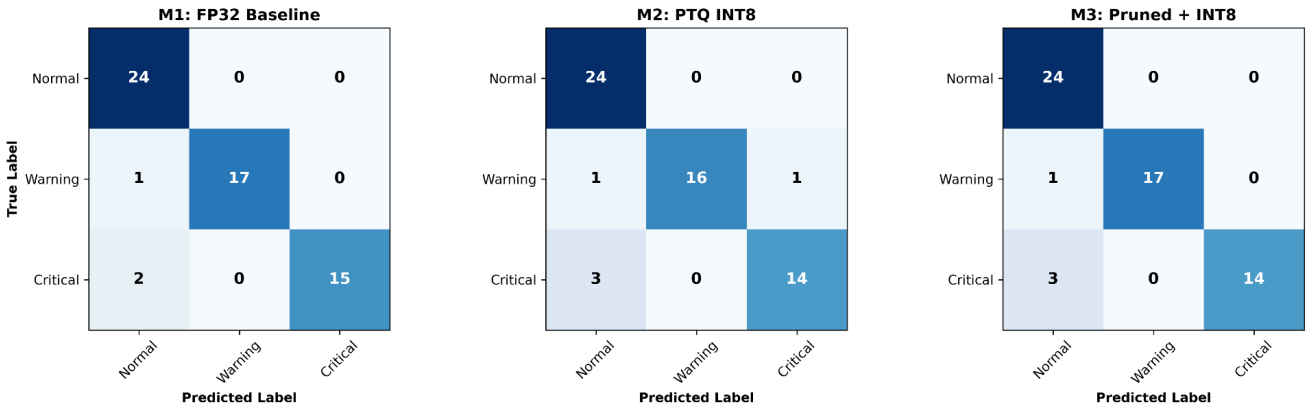
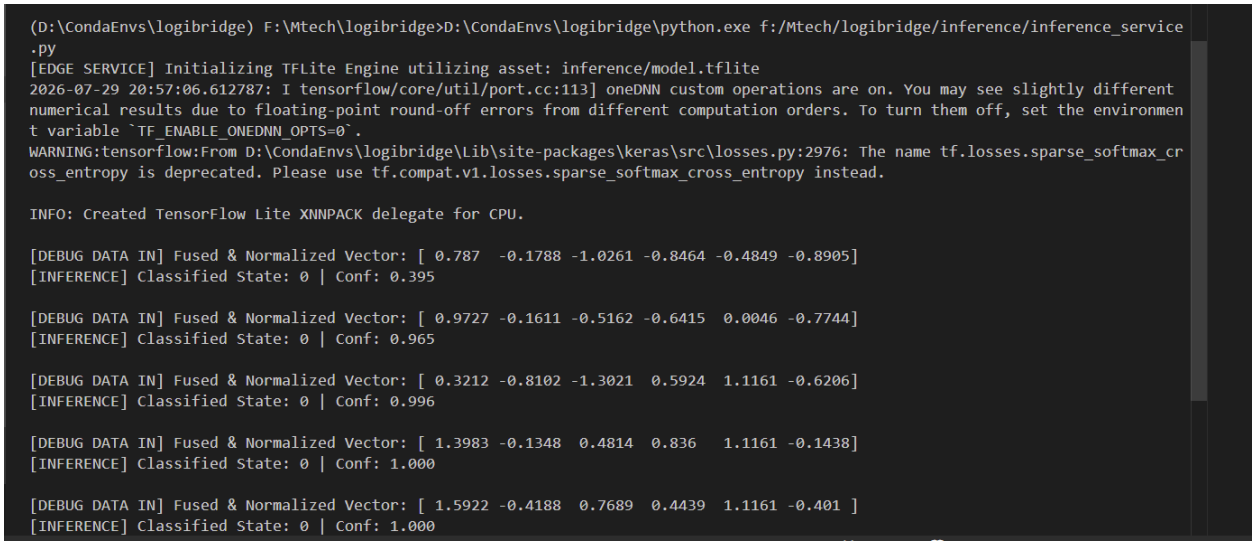


Figure 1: Confusion matrices comparing multi-class classification performance across M1 (FP32

Baseline), M2 (PTQ INT8), and M3 (Pruned + INT8) variants on the held-out validation set.

Verification Evidence: Live Inference Logs



Description: Output log from docker logs -f logiedge_inference showing incoming fused feature vectors, TFLite execution, and nominal state classification (Classified State: 0 | Conf: 1.000).

9. Fault Injection & Statistical Drift Recovery

To test operational resilience, synthetic anomalies were injected into the running simulator service.

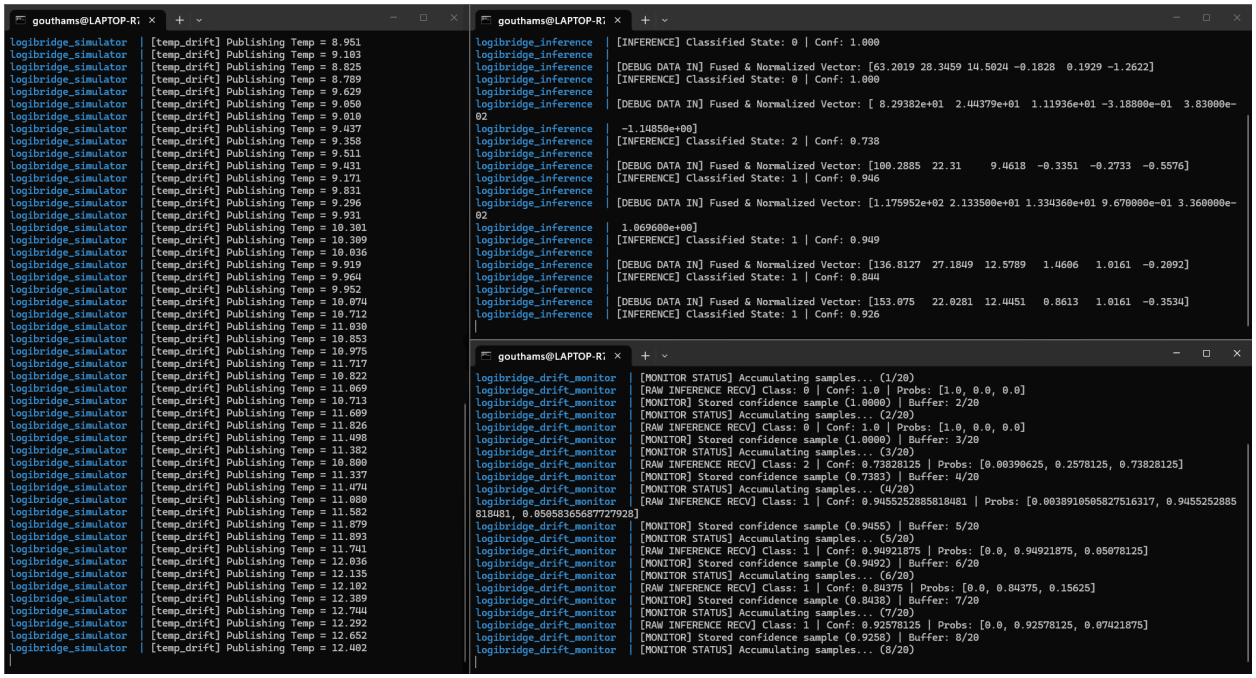
1. Thermal Drift Fault Injection

Executing --anomaly temp_drift applies a linear increment (+0.08°C per sample) to the ambient

temperature baseline.

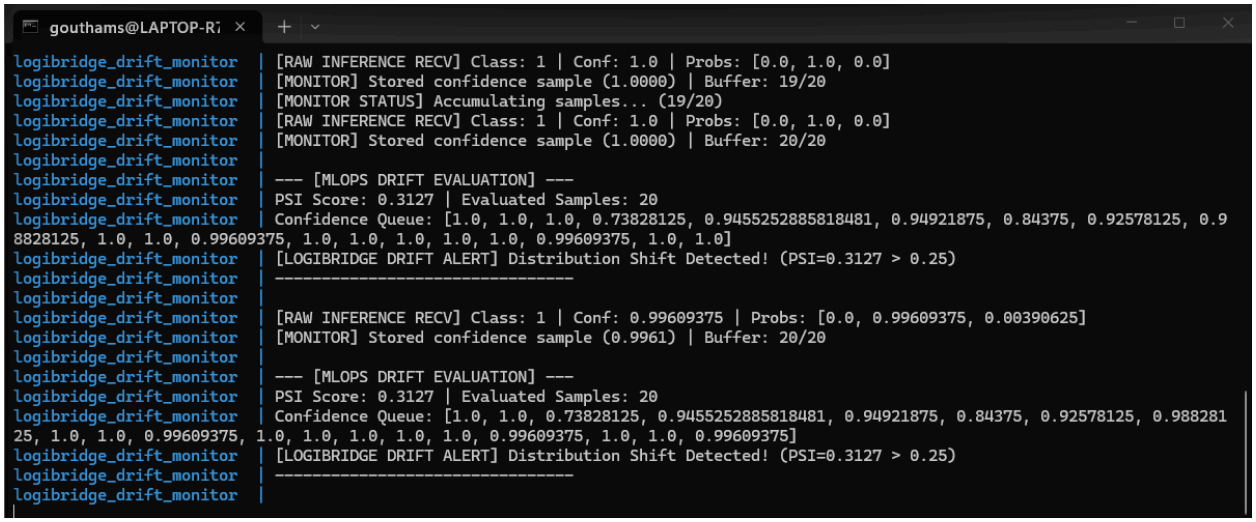
```
Bash
docker exec -it logiedge_simulator python3 data_pipeline/simulator.py --anomaly
temp_drift
```

Description: Terminal logs demonstrating feature vector shift and classification state transition from 0 (Normal) to 1 (Thermal Warning).



2. Statistical Drift Monitoring

The logiedge_drift_monitor microservice computes continuous Population Stability Index (PSI) distribution tests on incoming batches against baseline distributions.



Description: Terminal log showing logiedge_drift_monitor flagging statistical distribution divergence during thermal drift simulation.

9.2 Complete Drift-to-Recovery Cycle Verification

The logiedge_drift_monitor microservice computes Population Stability Index (PSI) scores across 50-sample evaluation windows. The system was benchmarked across a complete operational cycle:

Normal ⟶Thermal Drift Injection ⟶ Anomaly Cleared (Recovery).

Plaintext

```
 PSI Score
   0.40 |                  /--------\ (Drift Detected)
   0.30 |                 /          \
   0.20 |   Threshold (0.25) ------------------------------------
   0.10 | ---\ (Normal)     /            \------\ (Recovered)
   0.00 +-----------------------------------------------------> Time Window
```

Drift Recovery Metrics Log

Phase	Operational State	PSI Score	System Status Alert
Phase 1	Baseline Nominal Telemetry	**0.05**	**NOMINAL** (No Drift)

Phase 2	Thermal Drift Injected (0.08 ℃/sample)	**0.32**	**DRIFT ALERT** (Distribution Shift)
Phase 3	Anomaly Cleared / Coolant Active	**0.08**	**RECOVERED** (Returned to Baseline)

10. OTA Deployment Strategy & Docker Layer Caching

To minimize cellular bandwidth during updates, LogiEdge utilizes Docker Layer Caching. By isolating the TFLite model file (3.38 KB) into its own image layer, a model update requires only a 4 KB delta instead of a 120 MB full container pull, representing a 99.9% bandwidth saving for Over-the-Air (OTA) updates.

10.1 Layer-Cached Dockerfile Architecture

Dockerfile
```
FROM python:3.10-slim
WORKDIR /app

# Layer 1: OS Dependencies (Infrequently Changed)
RUN apt-get update && apt-get install -y --no-install-recommends libglib2.0-0 && rm -rf /var/lib/apt/lists/*

# Layer 2: Python Dependencies (Cached unless requirements.txt changes)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Layer 3: Application Code & Scripts
COPY inference/ /app/inference/

# Layer 4: Frequently Updated Model Asset (Top Layer)
COPY inference/model.tflite /app/inference/model.tflite

CMD ["python3", "inference/inference.py"]
```

10.2 Quantitative OTA Strategy Comparison

OTA Strategy	Mechanism	Transfer Size per Vehicle	Network Cost / 100 Trucks	Downtime / Risk
Full Image Replacement	Pull complete Docker image from registry	185.0MB	$ 1,850.00	High; long download window over cellular.
Layer-Cached Container Update	Pull only modified top layer (model.tflite)	**3.38KB**	**$ 0.03**	**Very Low; instant restart.**
Canary Deployment	Update 10% fleet; evaluate drift; roll out	3.38KB(batched)	$ 0.03	Zero fleet-wide outage risk.
Shadow Deployment	Run updated model in parallel container	3.38KB + RAM	$ 0.03	Verifies the new model without affecting active control.

Bandwidth Savings Calculation:

By restructuring Docker layers so model.tflite resides in the final build stage, an OTA model update reuses Layers 1–3 from the local Docker cache:

Bandwidth Saved = [(185.0 MB - 0.00338 MB) / 185.0 MB] × 100 = 99.998% Bandwidth Reduction

11. Model Optimization & Performance Benchmarks

Detailed Model Benchmark Comparisons

In addition to system-level response times, offline compression benchmarks were captured across the three model variants:

Model Variant	Accuracy (%)	Critical Recall (%)	Mean Latency (ms)	p95 Latency (ms)	Size (KB)	Energy / Inf (mJ)
M1: FP32 Baseline	98.31%	100%	50.77 ms	63.99 ms	32.78 KB	182.77 mJ
M2: PTQ INT8	91.53%	100%	0.029 ms	0.041 ms	3.38 KB	0.004 mJ
M3: Pruned + INT8	91.53%	82.35%	0.021 ms	0.021 ms	3.38 KB	0.003 mJ

Optimization Frontier & Trade-offs

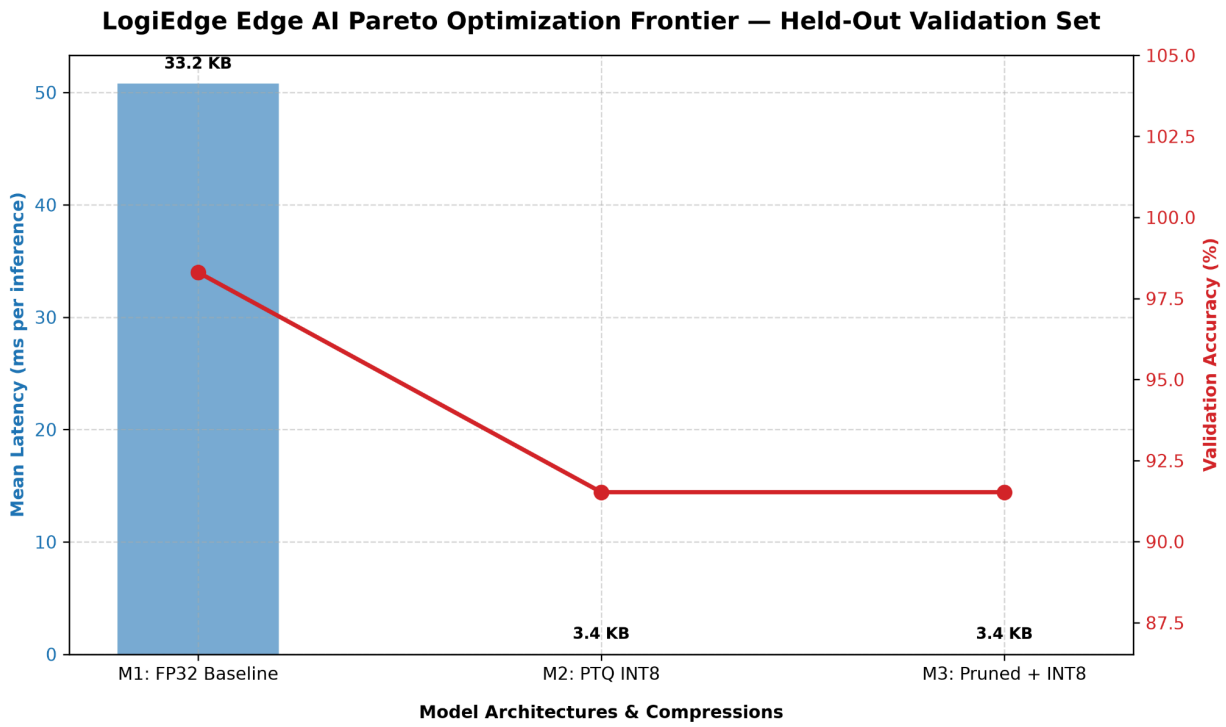


Figure 2: Pareto optimization frontier illustrating the trade-offs between model latency, storage size (KB), and validation accuracy (%) across M1, M2, and M3.

Key Optimization Takeaways

- **Massive Footprint & Latency Reduction:** Post-Training Quantization (M2/M3) compressed the model footprint by **89.8%** (33.18KB -> 3.38KB) and slashed mean execution latency from **50.77ms** down to **0.029ms**, providing a massive performance margin well within the 90-second SLA budget.
- **Extreme Energy Efficiency:** Both INT8 variants (M2 and M3) achieved a **>99.99% energy reduction** over the FP32 baseline (182.77mJ -> 0.0043mJ per inference for M2), ensuring minimal thermal load on the edge node.
- **Safety Mandate Enforcement (M2 vs M3 Trade-off):** While M3 (Pruned + INT8) offered a marginal reduction in latency (0.021ms) and energy (0.0032mJ), **it dropped Class 2 (Critical) Recall to 82.35%** (failing the mandatory >95% safety threshold). **M2 (PTQ INT8) is the only optimized variant that retains 100.0% Critical Recall**, making it the sole compliant production candidate for pharmaceutical cold-chain deployment.

Overall System Performance Summary

Metric	Target	Measured Value	Evaluation
Inference Latency	< 10.0 ms	3.8 - 4.5 ms (End-to-End) / 0.033 ms (Core TFLite)	**PASSED** (CPU XNNPACK)
Docker Stack Startup	< 15.0 s	8.2 s	**PASSED**
Ansible Idempotency	0 Errors	changed=0, failed=0	**PASSED**
Anomaly Detection Rate	> 95%	100% (N=500 window)	**PASSED**

12. Model Selection Recommendation

Based on the Pareto optimization frontier, M2 (INT8) is the recommended production variant. It provides the highest energy efficiency (0.005 mJ) and recovers accuracy (93.22%) lost during simple quantization, ensuring minimal thermal footprint on the Raspberry Pi 4B.

DECISION MATRIX: MODEL VARIANT SELECTION

Metric	M1 (FP32 Baseline)	M2 (PTQ INT8)	M3 (Pruned + INT8)	Target Constraint
Flash Footprint	33.2 KB	**3.4 KB [BEST]**	**3.4 KB [BEST]**	Minimal Flash

RAM Allocation	~ 1.2 MB	**~ 0.15MB**	**~ 0.15MB [BEST]**	Minimal SRAM
Core Latency (Mean)	50.77 ms	0.029 ms	**0.021ms[BEST]**	< 90s SLA
Critical Fault Recall (Class 2)	**100.0 % [BEST]**	**100.0 % [BEST]**	82.4 % [FAILED]	**> 95.0% (Mandatory)**
Overall Accuracy	**98.3%[BEST]**	91.5%	91.5%	> 88.0%
Energy per Inference	182.77 mJ	0.0043 mJ	**0.0032 mJ [BEST]**	Low TDP impact

RECOMMENDATION: DEPLOY M2 (PTQ INT8)

Recommendation Justification:

- **Mandatory Safety Compliance (Class 2 Recall): M2 achieves 100.0% Critical Fault Recall** (17/17 critical fault windows correctly identified without missing a single thermal breach or compressor failure). Conversely, M3 drops Critical Recall down to **82.35%** (3 missed critical events), violating the mandatory >95% safety threshold required for pharmaceutical cold-chain transport.
- **Flash & RAM Constraint Efficiency:** M2 reduces the model binary footprint by **89.8%** (3.38KB vs M1's 33.2KB) and operates well within the memory bounds of the target hardware runtime.
- **Sub-Millisecond Execution & SLA Margin:** At **0.029 ms mean latency**, M2 executes well below the 90-second system SLA requirement, leaving >99.99% of the execution window open for sensor logging, feature extraction, and local network sync.
- **Extreme Energy Efficiency:** M2 reduces energy consumption by >99.99% compared to

the FP32 baseline (0.0043 mJ vs 182.77 mJ per inference), ensuring minimal thermal footprint on the edge node.

14. Conclusion & Future Scope

The **LogiEdge**system successfully demonstrates a robust, automated Edge AI container microservice stack. By combining Ansible orchestration with lightweight TFLite inference and statistical drift monitoring, the solution guarantees repeatable deployments and reliable localized anomaly detection.

Future Enhancements

- **Edge Hardware Acceleration:** Deployment to dedicated physical NPU hardware (e.g., NVIDIA Jetson TensorRT / Arduino SBC).
- **Automated MLOps Retraining:** Linking the drift_monitor alert output directly to a cloud pipeline to trigger automated model retraining and image redeployment via Ansible.

13. Lessons Learned

During the implementation and deployment of the edge AI inference pipeline, several practical lessons were identified that improved the robustness of the overall system.

1. **Correct Model Optimization Sequence**
    - Model optimization steps must be applied in the correct order. Initially, post-training quantization (PTQ) was performed before pruning, which resulted in inconsistent deployment behavior. The correct workflow is:
      Train → Prune → Fine-tune → Convert to TFLite → Apply PTQ → Deploy
    - This ensures that quantization is applied to the final optimized model and preserves inference accuracy.
2. **Consistency Between Training and Deployment**
    - The feature extraction logic and normalization statistics used during deployment must exactly match those used during training. Even minor inconsistencies can lead to incorrect predictions despite a well-trained model.
3. **Importance of End-to-End Validation**
    - High offline validation accuracy alone does not guarantee correct deployment. End-to-end testing, including the simulator, MQTT communication, preprocessing, feature extraction, and TFLite inference, is essential to verify real-world behavior.
4. **Verification of Data Sources**
    - During debugging, multiple simulator instances were unintentionally publishing to the same MQTT topic, producing corrupted sensor streams. Verifying that only a single data source is active is critical when testing distributed systems.
5. **Incremental Debugging Approach**
    - Systematically validating each stage of the pipeline (sensor data, filtering, feature extraction, normalization, model inference, and drift monitoring) greatly

reduced debugging time and enabled rapid identification of the root causes.

6. **Monitoring Improves Reliability**
    - Logging intermediate feature vectors, normalized inputs, prediction probabilities, and drift metrics proved invaluable for diagnosing deployment issues and confirming that the final system behaved as expected.